

Mock Exam

CS 101 Algorithmic Problem Solving L7

Fall 2023

Name(s): _____

HU ID (e.g., xy01042): _____

Exam Instructions

- Exam Duration: 2 hours and 30 minutes
- Number of Questions: This exam contains 4 coding questions and 1 debugging question.
- Technical Requirements: You are not allowed to use code editors, compilers or online resources during this exam. Only electronic calculators are permitted during this exam.
- Communication: During the exam, do not communicate with anyone except the instructor or exam proctors if you have questions about the exam.
- Honesty: Academic honesty is expected. Do not engage in any form of cheating, plagiarism, or unauthorized collaboration.
- Answer Format: Write your answers clearly in the provided spaces.
- Time Management: Monitor the time carefully and pace yourself. Do not spend too much time on any one question or section.
- Exam Completion: Before submitting, take a moment to review your answers to ensure you have completed all questions and that your responses are clear and accurate. **Good luck!**

1. Packing The Important Things

You are leaving the country and your family is keen for you to take along the one thing you better not live without: Nihari. The good people at Nihari Inn can pack nihari for you in either spherical or cylindrical cans but not both for travel purposes. You are thinking to fill as many cans in your suitcase as you can and want to know which of the can types you should choose. Write a function named *max_cans* that takes parameters named *sphere_radius*, *cylinder_radius*, *cylinder_height*, and *suitcase_volume* and advises about the type of can to choose. Write helper functions, *sphere_volume* and *cylinder_volume* that take the necessary arguments in order to return the volume of a sphere and a cylinder respectively. *max_cans* must call these functions.

Hint

Volume of a sphere: $V_{\text{sphere}} = \frac{4}{3}\pi r^3$

Volume of a cylinder: $V_{\text{cylinder}} = \pi r^2 h$

Constraints

- *sphere_radius*, *cylinder_radius*, *cylinder_height*, and *suitcase_volume* are of numeric type (can be int or float)
- *sphere_radius* > 0, *cylinder_radius* > 0, *cylinder_height* > 0, *suitcase_volume* > 0

Interaction

The input consists of four space separated numbers representing *sphere_radius*, *cylinder_radius*, *cylinder_height*, and *suitcase_volume* respectively. The output should contain a string that states: “Take the cylindrical cans” or “Take the spherical cans” or “Take any type of can” depending on the *type* of the **maximum** number of cans that can fit in the suitcase.

Sample

Input	Output
5 10 1 2000	Take the cylindrical cans
5 10 2 2000	Take any type of can

In the first case sphere radius is 5, $V_{\text{sphere}} = \frac{4}{3}\pi r^3 = (4/3) * (22/7) * (5 * 5 * 5) = 523.8$. Then cylinder radius is 10, cylinder height is 1 and $V_{\text{cylinder}} = \pi r^2 h = (22/7) * (10 * 10) * (1) = 314.28$. Now the volume of suitcase is 2000. So number of sphere cans that can fit in the suitcase is $2000 / 523.8 = 3$ and number of cylindrical cans that can fit in suitcase is $2000 / 314.28 = 6$ hence the output is “Take the cylindrical cans”.

In the second case $V_{\text{sphere}} = \frac{4}{3}\pi r^3 = (4/3) * (22/7) * (5 * 5 * 5) = 523.8$. Then cylinder radius is 10, cylinder height is 2 and $V_{\text{cylinder}} = \pi r^2 h = (22/7) * (10 * 10) * (2) = 628.57$. Number of sphereical cans that can fit is $2000 / 523.8 = 3$ and number of cylindrical cans that can fit are $2000 / 628.57 = 3$ hence the output is “Take any type of can”.

Exercise

In the space provided, indicate the outputs for the given inputs.

Input	Output
4 10 2 2000	Take the spherical cans
6 12 2 5000	Take any type of can
3 6 0.5 1500	Take the cylindrical cans

Pseudocode

```
# Helper function to calculate the volume of a sphere
def sphere_volume(radius):
    return (4/3) * (22/7) * radius**3

# Helper function to calculate the volume of a cylinder
def cylinder_volume(radius, height):
    return (22/7) * radius**2 * height

# Main function to advise on the choice of cans
def max_cans(sphere_radius, cylinder_radius, cylinder_height, suitcase_volume)
    :
    # Calculate volumes of a spherical can and a cylindrical can
```

```
spherical_can_vol = sphere_volume(sphere_radius)
cylindrical_can_vol = cylinder_volume(cylinder_radius, cylinder_height)

# Calculate the maximum number of each type of can that can fit in the
# suitcase
max_spherical_cans = int(suitcase_volume // spherical_can_vol)
max_cylindrical_cans = int(suitcase_volume // cylindrical_can_vol)

# Determine the advice based on the maximum number of cans
if max_spherical_cans > max_cylindrical_cans:
    print("Take the spherical cans")
elif max_cylindrical_cans > max_spherical_cans:
    print("Take the cylindrical cans")
else:
    print("Take any type of can")
max_cans(sphere_radius, cylinder_radius, cylinder_height, suitcase_volume)
```

Dry Run

Using any **one** input provided in the **Exercise section** above, dry run your pseudocode in the space below.

Problem Identification

Briefly explain the underlying problem you identified in the above question that led you to your solution.

Input: *sphere_radius*, *cylinder_radius*, *cylinder_height*, and *suitcase_volume*

Output: Printed advice on which type of can to take based on calculating maximum number of cans that can fit in the suitcase.

2. Enormous Integer Overflow

You are working with a lot of data and you have N integers, you try to divide it using some K to figure out how many numbers are divisible by K .

This is an important task as the more numbers there are the more you get paid.

Given N and K return the count according to the prompt.

Constraints

- $1 \leq N, K \leq 10^7$

Interaction

The input begins with two positive space separated integers N and K . The next N lines contains one positive integer.

The output should contain a single number denoting how many integers are divisible by K

Sample

Input	Output
3 3	2
1	
51	
966369	
1 3	1
3	

In the first case, $N = 3$ and $K = 3$. Out of the three inputs 1, 51 and 966369, only **2** numbers (51 and 966369) are divisible by 3, hence the output is 2.

In the second case, $N = 1$ and $K = 3$. 3 is divisible by 3 so the output is 1.

Exercise

In the space provided, indicate the outputs for the given inputs.

Input	Output
2 2	1
32	
37	
6 8	4
72	
64	
8	
24	
18	
36	

Pseudocode

```
N = int(input())
k = int(input())
count = 0
for i in range(N):
    m = int(input())
    if m % k == 0:
        count = count + 1
print(count)
```

Dry Run

Using any two of the inputs provided in the Exercise section above, dry run your pseudocode

in the space below.

Problem Identification

Briefly explain the underlying problem you identified in the above question that led you to your solution.

Input: N, k

Output: Determine how many numbers out of N numbers are divisible by k .

3. Design the Pattern

Write an iterative function named *pattern* to generate a symmetrical pattern for a given parameter N .

Constraints

- $N \in \mathbb{Z}$
- $0 \leq N \leq 9$

Interaction

The input comprises a single line containing a single integer denoting the value of N .

The output must consist N lines, each containing the following pattern.

Sample

Input	Output
3	1 1 2 1 1 2 4 2 1
6	1 1 2 1 1 2 4 2 1 1 2 4 8 4 2 1 1 2 4 8 16 8 4 2 1 1 2 4 8 16 32 16 8 4 2 1

In the first case, $N = 3$. The output consists of 3 series of numbers printed on 3 separate lines.

In the second case, $N = 6$. The output consists of 6 separate lines with numbers in a particular pattern.

Exercise

In the space provided, indicate the outputs for the given inputs.

Input	Output
4	1
	1 2 1
	1 2 4 2 1
	1 2 4 8 4 2 1
5	1
	1 2 1
	1 2 4 2 1
	1 2 4 8 4 2 1
	1 2 4 8 16 8 4 2 1

Pseudocode

```
def pattern(n):
    for index in range(n):
        for i in range(index+1):
            print(2**i, end = ' ')
        for j in range(index - 1, -1, -1):
            print(2**j, end = ' ')
        print()
```

Dry Run

Using any **one** of the inputs provided in the **Exercise section** above, dry run your pseudocode in the space below.

Problem Identification

Briefly explain the underlying problem you identified in the above question that led you to your solution.

Input: n

Output: A triangular pattern of n lines each consisting of a palindrome.

4. Temperature Confusion

Fatima lives in Pakistan but her best friend Sana lives in United States. They both use different temperature system as United states doesn't use Celsius. Therefore, whenever they talk about weather there is great confusion. Fatima decides to make a function that takes temperature T and a letter S that indicates the unit temperature needs to be converted to.

Help her develop such a function called TempConverter which uses two helper functions to print the converted temperature and the system it is converted to. If $S = C$ then

FahrenheitToCelsius is used and if $S = F$ then CelsiusToFahrenheit is used. Helper functions just return the converted temperature.

Hint: Temperature in degrees Fahrenheit = (Temperature in degrees Celsius * 9/5) + 32.

Constraints

- $S = C$ or $S = F$
- $0 \leq T \leq 212$
- Round off the temperature to 2 decimal places.

Interaction

The input comprises a single line containing an integer denoting the value of T and a letter denoting the value of S .

The output must contain a line stating the converted temperature and the system it is converted to.

Sample

Input	Output
44 C	6.67 Degree Celsius is the temperature for Fatima.
0 F	32.00 Fahrenheit is the temperature for Sana.

In the first case, 44 Fahrenheit is to be converted to the celsius. Using the formula and rounding it off we get 6.67. Moreover, the conversion is to the degree celsius hence, it is for Fatima.

In the second case, 0 Degree Celsius is to be converted to Fahrenheit. The given formula is used to obtain the temperature in Fahrenheit and as it is not in celsius we know it is for Sana.

Exercise

In the space provided, indicate the outputs for the given inputs.

Input	Output
100 F	212.00 Fahrenheit is the temperature for Sana
98 C	36.67 degree Celsius is the temperature for Fatima
56 F	132.80 Fahrenheit is the temperature for Fatima

Problem Identification

Briefly explain the underlying problem you identified in the above question that led you to your solution.

Input: T,S

Output: Temperature conversion to the appropriate unit based on the string S.

Pseudocode

```
def FahrenheitToCelsius(t):
    converted = (t-32)*(5/9)
    return round(converted, 2)

def CelsiusToFahrenheit(t):
    converted = (t*(9/5)+32)
    return round(converted, 2)
```

```
def TempConverter(T,S):
    if S == "C":
        print(FahrenheitToCelsius(T), "degree Celsius is the temperature
                                                    for Fatima")
    else:
        print(CelsiusToFahrenheit(T), "Fahrenheit is the temperature for
                                                    Fatima")

TempConverter(T,S)
```

Dry Run

Using any **one** of the inputs provided in the **Exercise section** above, dry run your pseudocode in the space below.

5. Highest Divisor

You are given an integer N .

Write a function `highest` which takes in N and returns the highest divisor between 1 and 10 (inclusive) which divides N .

Make sure to call your function and to present the output use:

`print(functionname(parameters))`

Constraints

- $2 \leq N \leq 100000$
- You are only allowed to use a while loop for this question.

Interaction

The input comprises a single integer which is the value of N

The output must contain a single integer which is the highest divisor.

Sample

Input	Output
91	7
24	8

In the first case, $N = 91$, the divisors are 1, 7, 13, 91 only 1 and 7 are in the range and so the answer is 7.

In the second case, $N = 24$, the divisors are 1, 2, 3, 4, 6, 8, 12, 24 out of which 1, 2, 3, 4, 6 and 8 are in the range so the answers is 8.

Proposed Solution

```
1 def highest(N):
2     divisor = 10
3     while divisor >= 1:
4         if N % divisor != 0:
5             return divisor
6     divisor += 1
7     highest(N)
```

Dry Run

Using one of inputs provided in the Sample section above, dry run the proposed code solution below.

$N = 91$ The function call is indented incorrectly (it is inside the function definition) so there will be an error and the function will not be called.

Suppose the indentation was correct, and the function was called with value $N = 91$. $\text{divisor} = 10$, while $\text{divisor} \geq 1$ (True)

$N \% \text{divisor}$ not equal to 0 (True)

return $\text{divisor} - 10$ will be returned. Function exits.

So no matter which N is used, if N is not divisible by 10, then 10 will be returned (always). In case N is divisible by 10, there will be an infinite loop. The variable divisor will keep on incrementing, of course it will remain ≥ 1 and the while loop will never end. Nothing will be returned.

Error Identification

Briefly explain the errors you identified in the proposed code solution. Mention the line number and the errors in each line.

- $N \% \text{divisor}$ should be $== 0$ (because we need to check for exact divisibility)
- divisor should decrement (decrease by 1) rather than increase to avoid infinite loop.
- Line 6 needs to be indented inside the while loop so the decrement happens inside the loop
- function call needs to be indented outside the function definition

Correct Solution

Rewrite the lines of code you mentioned above with their errors corrected.

```
def highest(N):  
    divisor = 10  
    while divisor >= 1:  
        if N % divisor == 0:  
            return divisor  
        divisor -= 1  
print(highest(N))
```

Rough Work